

Electricity Consumption Forecasting with Recurrent Architectures

LSTM Baseline - Progress Report

Erdem YILMAZ
Adana Alparslan Türkeş Science and Technology University
Department of Artificial Intelligence Engineering

August 12, 2026

1 GitHub Repository

The repository for this project is available at:

- <https://github.com/eerdemYilmaz/electricity-consumption-forecasting>

2 Dataset

2.1 Source

The dataset used in this study is the *Power Consumption of Tetouan City* dataset.

- Kaggle: <https://www.kaggle.com/datasets/fedesoriano/electric-power-consumption>
- UCI Machine Learning Repository: <https://archive.ics.uci.edu/dataset/849/power+consumption+of+tetouan+city>

2.2 Related Publication

A paper associated with this dataset was found:

A. Salam and A. E. Hibaoui, “Comparison of Machine Learning Algorithms for the Power Consumption Prediction: — Case Study of Tetouan city —,” *2018 6th International Renewable and Sustainable Energy Conference (IRSEC)*, Rabat, Morocco, 2018, pp. 1–5, doi: 10.1109/IRSEC.2018.8703007.

2.3 Original Features

The original dataset contains the following variables:

- Temperature
- Humidity
- Wind Speed
- General Diffuse Flows
- Diffuse Flows

- Power Consumption — Zone 1
- Power Consumption — Zone 2
- Power Consumption — Zone 3

Only the first power consumption feature (**Zone 1**) is used in this study.

2.4 Data Used in This Study

Property	Value
Shape	52416×3
datetime	year, month, day
time	10-minute stride, starting at 00:00; next day follows 23:50 + 10
consumption	consumption value, e.g. 34.0556962 kWh
Date range	2025-07-01 to 2026-06-29 (nearly one year)
Scaling / normalization	None applied

Table 1: Structure of the data used for training.

2.5 Date Modification

The dates in the current data differ from the original dataset. The original date range was **1/1/2017 to 31/12/2017**. The date component was dropped by the following preprocessing step, which keeps only the hour and minute:

```
df["time"] = df["time"].apply(lambda x: pd.to_datetime(x).strftime("%H:%M"))
```

3 Model Architecture and Training Setup

Parameter	Value
Sequence length	60
Input size	1
Hidden size	64
Stacked layers	2
Batch size	64
Shuffle	False
Loss function	MSELoss
Optimizer	Adam
Learning rate	1×10^{-3}
Epochs	10

Table 2: Model and training hyperparameters.

3.1 Train / Test Split

Split	Shape (samples, sequence length, features)	Ratio
Train	$41884 \times 60 \times 1$	0.8
Test	$10472 \times 60 \times 1$	0.2

Table 3: Train and test set sizes.

4 Training Results

Epoch	Train Loss
1	248.0716
2	49.5510
3	20.2190
4	1.9236
5	0.7210
6	0.4287
7	0.3171
8	0.2547
9	0.2266
10	0.2154

Table 4: Training loss per epoch (10 epochs).

5 Metrics

Metric	Result
MSE	0.1610
RMSE	0.4013
MAE	0.2749
MAPE	0.98%
R2	0.9958

Table 5: Baseline test set metrics.

These values should be read against the persistence reference baseline reported in Section 6: a predictor that simply repeats the last observed value reaches $R^2 = 0.9920$ and $\text{MAPE} = 1.27\%$ on this dataset without any training. On a series with 10-minute resolution and high autocorrelation, an R^2 close to 0.99 is therefore not by itself evidence of a strong model; only the margin over the persistence baseline is informative.

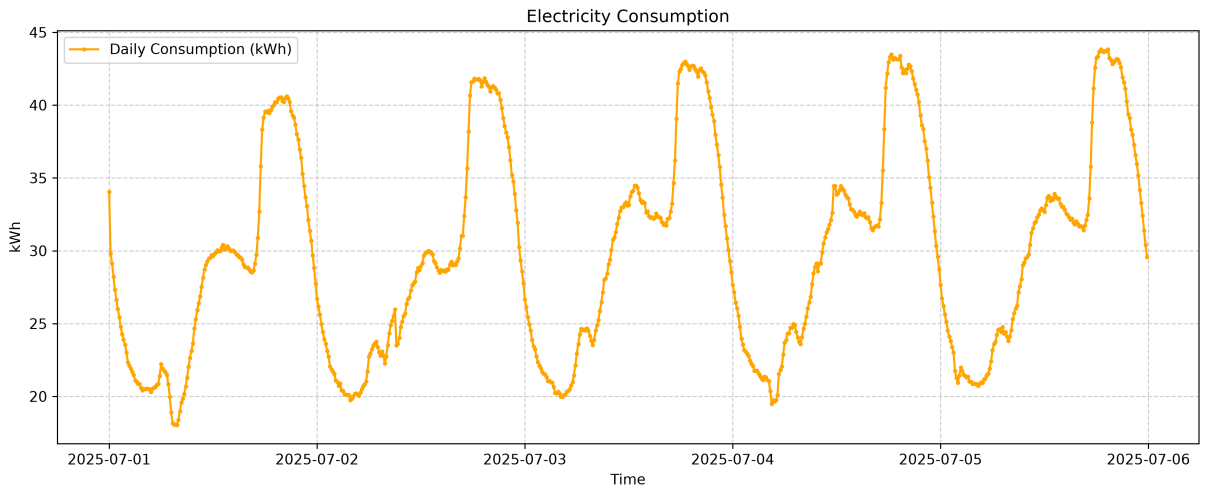


Figure 1: Electricity consumption.

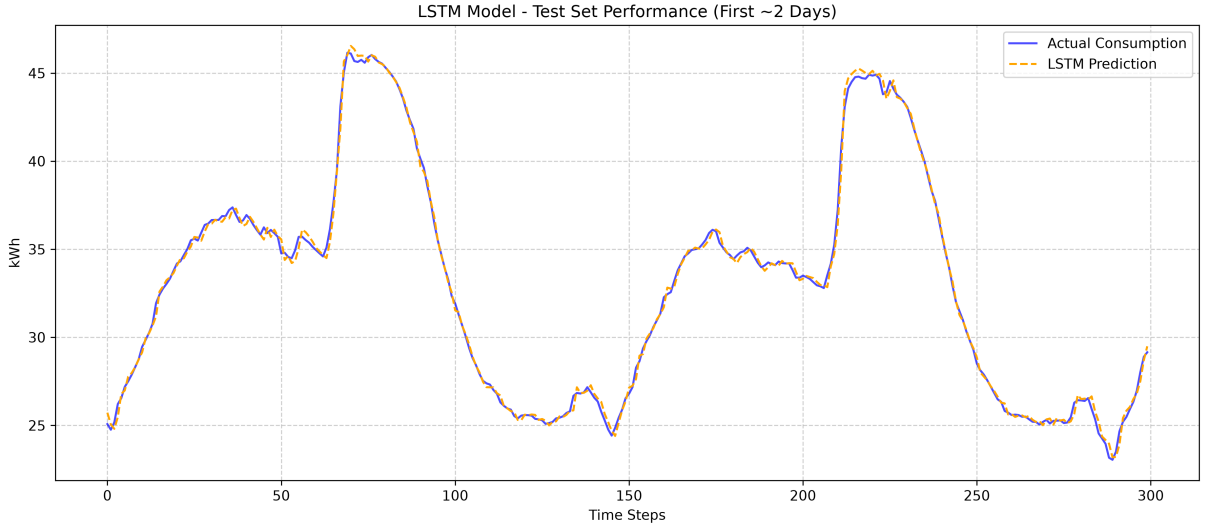


Figure 2: LSTM model — test set performance.

6 Ablation Study: Validation, Early Stopping and Scaling

6.1 Motivation

Three components were missing from the baseline setup: a validation split, early stopping, and input scaling. Adding them simultaneously would make it impossible to attribute any change in the metrics to a specific cause. The experiments below therefore follow a single-variable ablation design: two runs share the same seed, architecture, optimizer, epoch budget and early stopping policy, and differ only in whether standardization is applied.

6.2 Methodology

- **Fixed seed.** `random`, `numpy` and `torch` are all seeded with 42. Without this, two runs of identical code produce different loss curves and no comparison is meaningful.
- **Leakage-free chronological split.** The raw series is split **before** windowing into 70% / 10% / 20% partitions, and the sliding windows are then generated independently inside each partition. If the windows were generated first and split afterwards, windows near a boundary would take their inputs from one partition and their target from the next, leaking training timesteps into validation and test.
- **Persistence baseline.** A training-free reference predictor, $\hat{y}_t = y_{t-1}$, evaluated on the same partitions. It establishes the error floor that any trained model must beat to demonstrate value.
- **Early stopping.** Maximum 50 epochs, patience of 8 epochs on validation loss. The weights of the best validation epoch are restored before the test set is evaluated.
- **Scaling applied to Run B only.** Mean and standard deviation are computed on the training partition alone and then applied to validation and test. Fitting the scaler on the full series would leak test-set statistics into preprocessing.
- **Metrics in original units.** For Run B the predictions and targets are inverse-transformed before the metrics are computed, so Run A and Run B remain directly comparable in kWh.

Partition	Raw index range	Length	Sequences
Train	[0, 36691)	36691	36631
Validation	[36691, 41932)	5241	5181
Test	[41932, 52416)	10484	10424

Table 6: Leakage-free 70/10/20 chronological split. Each partition loses `sequence_length` = 60 samples to windowing.

6.3 Results

Model	MSE	RMSE	MAE	MAPE	R2
Persistence baseline	0.3032	0.5507	0.3697	1.27%	0.9920
Run A — no scaling	0.1820	0.4266	0.3278	1.18%	0.9952
Run B — standardized	0.1638	0.4047	0.2942	1.04%	0.9957

Table 7: Test set metrics, original kWh units. The test partition is evaluated once, after training and model selection are complete.

Model	MSE	RMSE	MAE	MAPE	R2
Persistence baseline	0.3824	0.6184	0.4162	1.23%	0.9909
Run A — no scaling	0.1123	0.3351	0.2330	0.72%	0.9973
Run B — standardized	0.1018	0.3191	0.2169	0.67%	0.9976

Table 8: Validation set metrics at the selected (best) epoch. These values are optimistically biased, because the epoch was chosen by minimising validation loss.

Run	Epochs run	Best epoch	Stopped early
Run A — no scaling	46	38	yes
Run B — standardized	32	24	yes

Table 9: Early stopping behaviour (maximum 50 epochs, patience 8).

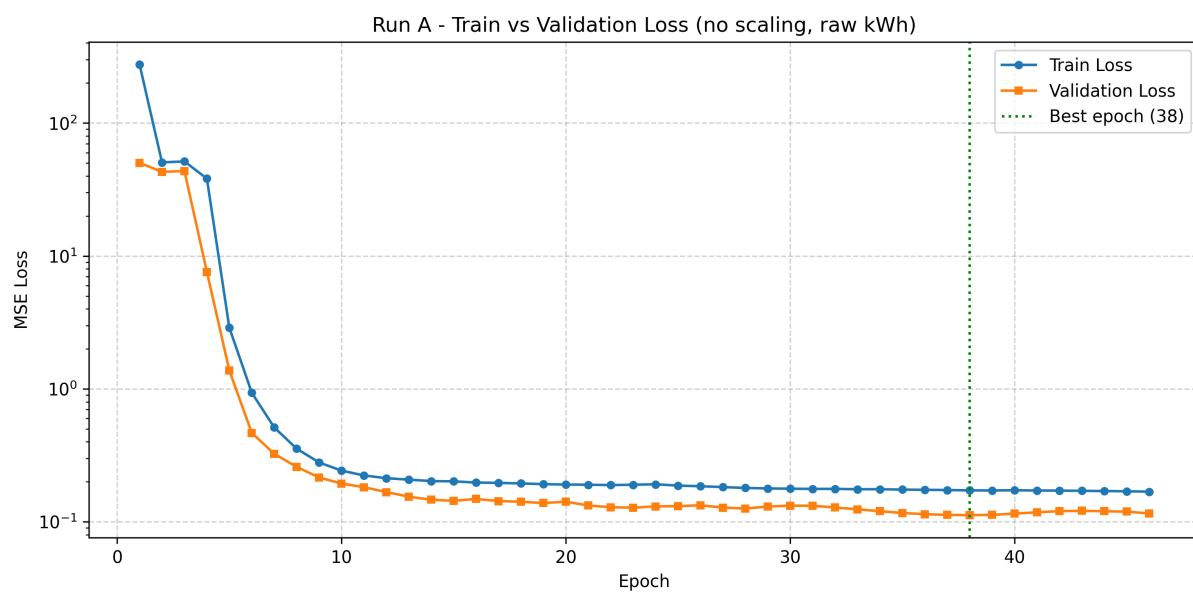


Figure 3: Run A — train and validation loss per epoch, no scaling.

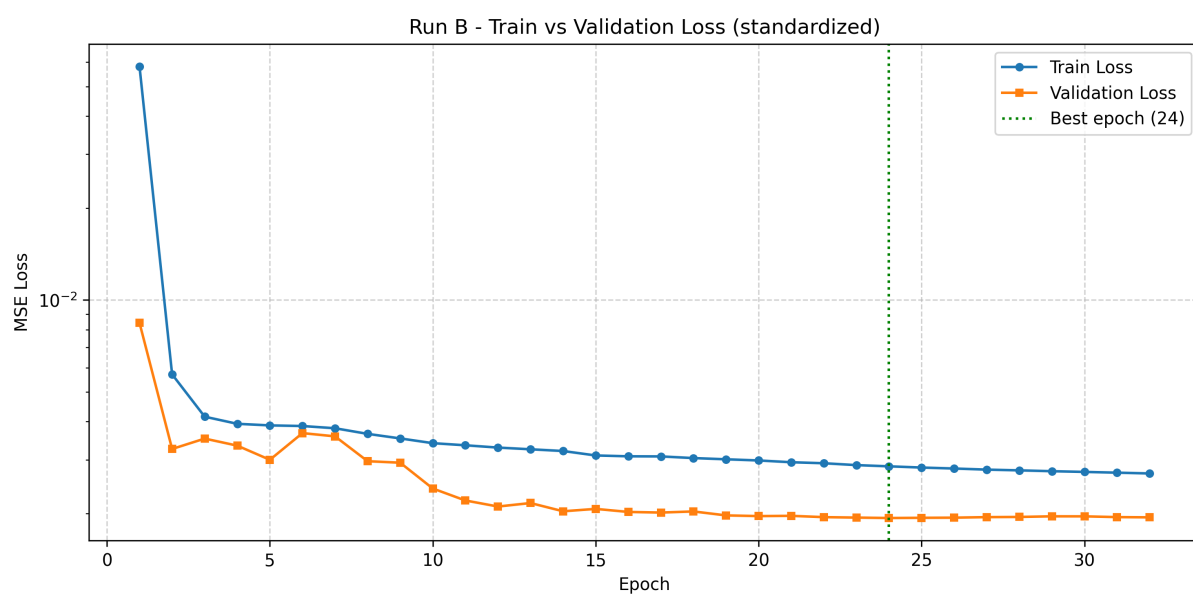


Figure 4: Run B — train and validation loss per epoch, standardized inputs.

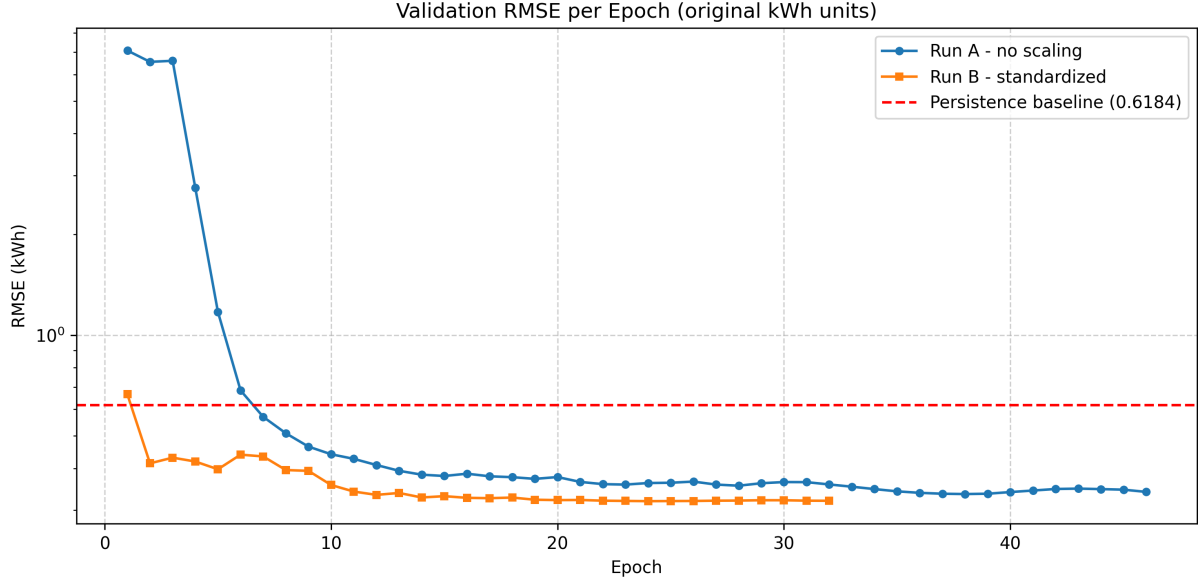


Figure 5: Validation RMSE per epoch for both runs, in original kWh units, against the persistence baseline.

6.4 Findings

1. **The persistence baseline is a strong reference on this dataset.** Without any training it reaches $R^2 = 0.9920$ and $\text{MAPE} = 1.27\%$ on the test partition. Consecutive 10-minute consumption values differ by very little relative to their magnitude (mean consumption ≈ 33.8 kWh, mean absolute change ≈ 0.42 kWh), so near-perfect R^2 is a property of the task, not of the model.
2. **Both trained models beat the baseline by a clear margin.** Run A reduces test RMSE by 22.5% relative to persistence and Run B by 26.5%. The model is therefore not simply reproducing the previous value.
3. **The main effect of scaling is convergence speed, not final quality.** Run B reaches a validation RMSE of 0.4139 at epoch 2, a level Run A only reaches at epoch 12 — roughly a sixfold difference in the number of optimizer steps required. After both runs are trained to convergence with early stopping, the remaining gap in test RMSE is only 5.1% (0.4266 versus 0.4047). An earlier version of this experiment used a fixed 10-epoch budget and reported a much larger gap (0.5318 versus 0.4318); that difference was an artefact of truncating training before the unscaled run had converged.
4. **Overfitting appears only once the epoch budget is large enough to expose it.** With a 10-epoch budget the validation loss never diverged from the training loss, so no overfitting signature was observable. With a 50-epoch budget the validation loss reaches its minimum at epoch 38 in Run A and epoch 24 in Run B and then rises slowly while the training loss continues to fall. The effect is mild — Run A validation loss increases from 0.1123 to 0.1157 between epochs 38 and 46 — and early stopping restores the best weights, but it confirms that a fixed epoch count is not a safe training policy here.
5. **The validation–test gap is substantial and expected.** Run B reports RMSE 0.3191 on validation but 0.4047 on test. Two effects contribute: the selected epoch was chosen by minimising validation loss, which biases the validation score optimistically, and the test partition is chronologically later, so seasonal drift is present. Neither can be measured without a test partition that is held out from all model-selection decisions.

7 Outputs

- Figures saved at `dpi = 300`.
- Baseline hyperparameters and metrics saved to `hyperparameter_logs.json`.
- Baseline model saved to `model/lstm_model_1.pth`.
- Ablation study implemented in `lstm_ablation.py`, with per-epoch history written to `hyperparameter_logs.json` under the experiment names `persistence_baseline`, `lstm_v4_noscale_earlystop` and `lstm_v5_scaled_earlystop`.
- Ablation models saved to `model/lstm_v4_noscale_earlystop.pth` and `model/lstm_v5_scaled_earlystop.pth`.

8 Planned Work

- GRU
- BiGRU
- Attention
- TCN
- Newer attention architectures (Kimi K3, DeepSeek V3)
- Sequential data modeling
- Multivariate input (temperature, humidity, wind speed, diffuse flows, Zones 2–3) as a prerequisite for Explainable Artificial Intelligence (XAI) based feature selection

9 Open Questions and Missing Parts

9.1 Resolved

- **Numeric precision.** Resolved: the `consumption` column is stored as `float64` (8 bytes) in both the parquet and the CSV file. The 4-byte (`float32`) representation appears only at training time, through `torch.tensor(..., dtype=torch.float32)`. The `datetime` column is stored as a string (`object`), not as `datetime64`.
- **No validation set.** Resolved: a leakage-free 70/10/20 chronological split is now used.
- **No early stopping.** Resolved: patience-based early stopping on validation loss, with restoration of the best weights.
- **No validation loss per epoch.** Resolved: training loss, validation loss and validation RMSE in original units are recorded for every epoch and plotted.
- **No scaling or normalization.** Resolved as an ablation: Run B applies standardization fitted on the training partition only, and its effect is quantified in Section 6.

9.2 Still Open

- **Attention and residual types.** Which attention and residual variants should be used (Kimi K3, DeepSeek V3)?
- **Single feature only.** Only Zone 1 power consumption is used; temperature, humidity, wind speed, general diffuse flows, diffuse flows and Zones 2–3 are unused.
- **Seasonal drift.** The margin over the persistence baseline shrinks on the chronologically later test partition. The cause has not yet been isolated.
- **Single-seed results.** All runs use seed 42. Repeating each configuration over several seeds would separate real differences from initialisation noise.
- **Horizon.** The current task is single-step ahead (10 minutes). Multi-step horizons have not been evaluated.

10 Discussion Notes

This section records the questions raised during the analysis, the hypotheses proposed, and the conclusions reached.

Q1. Are all consumption values stored in 4 bytes?

No. Both `data/processed_powerconsumption.parquet` and `data/data_processed.csv` store consumption as `float64`, that is 8 bytes per value. The conversion to `float32` happens only when the tensor is created for training. The `datetime` column is a string rather than a `datetime` type, so it is re-parsed on every read.

Q2. Is it reasonable to lower the learning rate because the unscaled values are large?

No, and the reasoning does not apply to this optimizer. Adam’s update is $\Delta\theta = \eta \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon)$. Multiplying all gradients by a constant scales $\hat{m}$ and $\sqrt{\hat{v}}$ equally, so the step size is unchanged; Adam is invariant to the global gradient scale. The argument would be correct for plain SGD, where $\Delta\theta = \eta \cdot g$ scales directly with the gradient magnitude.

The actual problem is the distance the output must travel. The network output starts near zero and the targets are centred around 33.8 kWh, while Adam bounds each parameter step at approximately $\eta = 10^{-3}$. Lowering the learning rate shortens each step and therefore makes the problem worse. Standardization is the correct remedy, and Run B confirms it: the same quality is reached in roughly one sixth of the optimizer steps.

Q3. Does the model appear to have memorised the data?

Not in the classical sense of overfitting. With the original 10-epoch budget the validation loss never diverged from the training loss, so the overfitting signature was absent. Two separate issues were identified instead. First, the visually near-perfect overlap between prediction and ground truth in Figure 2 is largely a property of the task: a persistence predictor produces a similar-looking curve. Second, once the epoch budget was extended to 50, a mild overfitting onset did become visible after epoch 38 (Run A) and epoch 24 (Run B), which is what early stopping now handles.